National University of Computer & Emerging Sciences, Karachi
Computer Science Department
Spring 2026, Lab Manual – 06

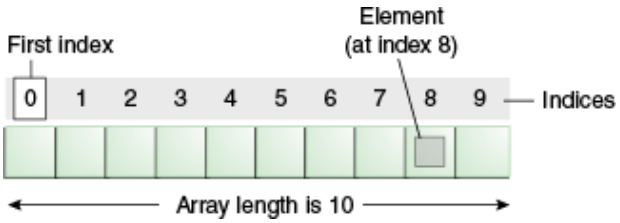
| | |
|---|---|
| Course Code: CS-1004 | Course: Object Oriented Programming Lab |
| Instructor : | Muhammad Owais |

# LAB - 6

# Arrays & Array list

# Arrays:

Java array is a construct which contains elements of a similar data type. Additionally, the elements of an array are stored in a contiguous memory location. It is a data structure where we store similar elements. We can store only a fixed set of elements in a Java array. Array in Java is index-based, the first element of the array is stored at the 0th index, 2nd element is stored on 1st index and so on.




# Types of Arrays

1. Single Dimensional Array
2. Multi-dimensional Array

# Single Dimensional Array

**Syntax for Declaration**

**Option 1:** dataType[] arr;
**Option 2:** dataType []arr;
**Option 3:** dataType arr[];

**Syntax for Instantiation**

arr = **new** datatype[size];

**Example**

```
public class Test {
    public static void main(String args[])
    {
        int a[]=new int[5];//declaration and instantiation
        a[0]=10;//initialization
        a[1]=20;
        a[2]=70;
        a[3]=40;
        a[4]=50;
        //traversing array
        for(int i=0;i<a.length;i++)
            System.out.println(a[i]);
    }
}
```

## Multi Dimensional Array

In such case, data is stored in row and column based index (also known as matrix form).

**Syntax for Declaration**

dataType[][] arrayRefVar; (or)

dataType [][]arrayRefVar; (or)

dataType arrayRefVar[][]; (or)

dataType []arrayRefVar[];

**Syntax for Instantiation**

int[][] arr=new int[3][3]; //3 row and 3 column

**Example**

```
public class Test{
    public static void main(String args[])
    {
    //declaring and initializing 2D array
        int arr[][]={{1,2,3},{2,4,5},{4,4,5}};
    //printing 2D array
        for(int i=0;i<3;i++){
            for(int j=0;j<3;j++){
                System.out.print(arr[i][j]+" ");
            }
            System.out.println();
        }
    }}
```

## Passing Arrays to Functions:

We can pass the java array to method so that we can reuse the same logic on any array.

**Example**

```java
public class Test {
    public static void main(String args[])
    {
        int a[]={33,3,4,5}; //declaring and initializing an array
        min(a); //passing array to method
    }
    static void min(int arr[])
    {
        int min=arr[0];
        for(int i=1;i<arr.length;i++)
            if(min>arr[i])
                min=arr[i];

        System.out.println(min);
    }
}
```

## Java ArrayList

**ArrayList** class uses a *dynamic* array for storing the elements. It is like an array, but there is *no size limit*. We can add or remove elements anytime. So, it is much more flexible than the traditional array. It is found in the *java.util* package.

- ArrayList class can contain duplicate elements.
- ArrayList class maintains insertion order.
- ArrayList allows random access because array works at the index basis.

### Commonly used Constructors

- ArrayList( ): It is used to build an empty array list.
- ArrayList (int capacity): It is used to build an array list that has the specified initial capacity.

### Common Methods of ArrayList

| | |
|---|---|
| add(**value**) | appends value at end of list |
| add(**index**, **value**) | inserts given value just before the given index, shifting subsequent values to the right |
| E get(**index**) | returns the value at given index |
| E remove(**index**) | removes/returns value at given index, shifting subsequent values to the left |
| E set(**index**, **value**) | replaces value at given index with given value, returns element that was previously at index. |
| int size() | returns the number of elements in list |
| toString() | returns a string representation of the list such as "[3, 42, -7, 15]" |

## Additional Methods of ArrayList

| | |
|---|---|
| addAll(**list**)<br>addAll(**index**, **list**) | adds all elements from the given list to this list<br>(at the end of the list, or inserts them at the given index) |
| contains(**value**) | returns true if given value is found somewhere in this list |
| containsAll(**list**) | returns true if this list contains every element from given list |
| equals(**list**) | returns true if given other list contains the same elements |
| iterator()<br>listIterator() | returns an object used to examine the contents of the list<br>(seen later) |
| lastIndexOf(**value**) | returns last index value is found in list (-1 if not found) |
| remove(**value**) | finds and removes the given value from this list |
| removeAll(**list**) | removes any elements found in the given list from this list |
| retainAll(**list**) | removes any elements *not* found in given list from this list |
| subList(**from**, **to**) | returns the sub-portion of the list between<br>indexes **from** (inclusive) and **to** (exclusive) |
| toArray() | returns the elements in this list as an array |

- *You have to import java.util.* to use ArrayList*
- *E refers to type of elements in the above methods*

## Example

```
import java.util.*;
public class Test{
    public static void main(String args[]){
        ArrayList<String> list=new ArrayList<String>();//Creating arraylist
        list.add("Mango");//Adding object in arraylist
        list.add("Apple");
        list.add("Banana");
        list.add("Grapes");
        //Printing the arraylist object
        System.out.println(list);
    }
}
```

```java
import java.util.*;

ArrayList<Integer> list = new ArrayList<>(Arrays.asList(10, 20, 30, 40));

list.add(50);                      // adds 50 at end
list.add(1, 15);                   // inserts 15 at index 1
list.get(2);                       // returns element at index 2
list.remove(3);                    // removes element at index 3
list.set(0, 5);                    // replaces index 0 value with 5
list.size();                       // returns total number of elements
list.toString();                   // returns list as string

list.addAll(Arrays.asList(60,70));        // adds multiple elements at end
list.addAll(2, Arrays.asList(25,26));     // inserts multiple at index 2
list.contains(20);                       // checks if 20 exists
list.containsAll(Arrays.asList(5,15));   // checks if all given elements exist
list.equals(new ArrayList<>(list));      // compares two lists
list.iterator();                        // returns iterator object
list.listIterator();                    // returns list iterator
list.lastIndexOf(20);                   // returns last index of 20
list.remove(Integer.valueOf(15));       // removes value 15
list.removeAll(Arrays.asList(25,26));   // removes all given values
list.retainAll(Arrays.asList(5,20));    // keeps only given values
list.subList(1,3);                      // returns sublist from index 1 to 2
list.toArray();                         // converts list to array
```

## Iterating ArrayList using Iterator

```java
import java.util.*;
public class Test{
    public static void main(String args[]){
        ArrayList<String> list=new ArrayList<String>();//Creating arraylist
        list.add("Mango");//Adding object in arraylist
        list.add("Apple");
        list.add("Banana");
        list.add("Grapes");
        //Traversing list through Iterator
        Iterator itr=list.iterator();//getting the Iterator
        while(itr.hasNext()){//check if iterator has the elements
            System.out.println(itr.next());//printing the element and move to next
        }
    }
}
```

## Example: Sorting ArrayList

```
import java.util.*;
public class Test{
    public static void main(String args[]){
        //Creating a list of fruits
        List<String> list1=new ArrayList<String>();
        list1.add("Mango");
        list1.add("Apple");
        list1.add("Banana");
        //Sorting the list
        Collections.sort(list1);
        //Traversing list through the for-each loop
        for(String fruit:list1)
            System.out.println(fruit);
    }
}
```